Vorabfragen

- Ein Roboter hat einen Lidar-Sensor, der ständig Messwerte liefert. Ein anderes Programm soll diese Daten laufend auswerten. Wie sollten die beiden Programme Ihrer Meinung nach kommunizieren?
  a) Das auswertende Programm fragt bei Bedarf nach dem aktuellen Wert (Anfrage–Antwort).
  b) Der Sensor sendet kontinuierlich Daten, und alle Interessierten „hören mit“.
  c) Beide Programme schreiben in eine gemeinsame Datei.
  d) Beide laufen im selben Programm, damit keine Kommunikation nötig ist.

- Ein Roboter soll eine 10 Minuten dauernde Aufgabe ausführen, z. B. „Fahre in die Küche“. Nach 3 Minuten bemerken Sie, dass das Ziel falsch war. Was müsste die Kommunikation zwischen Steuerung und Roboter können, damit Sie die Aufgabe abbrechen und zwischendurch den Fortschritt sehen können?

- Ihr Roboter besteht aus vielen Teilen: Motorsteuerung, Lidar, Kamera, Navigation, ... Wie würden Sie diese Software aufteilen: ein großes Programm oder viele kleine? Nennen Sie Vor- und Nachteile.



Cyber-Physische Systeme – WS 26/27

Gelesen von Prof. Dr. Daniel Gaida

Inhalt

Welche Eigenschaften sollte ein Betriebssystem für CPS erfüllen?

Erste Schritte mit ROS (Robot Operating System):

- Grund: Auf der Hardware im Labor (Turtlebot, LIMO Cobot, Niryo NED2, …) laufen ROS2 bzw. ROS
- ROS2

Welche Eigenschaften sollte ein Betriebssystem für CPS erfüllen?

Determinismus:

- Vorhersehbare Reaktion auf Ereignisse in einem festgelegten Zeitrahmen.
-
-
-

Welche Eigenschaften sollte ein Betriebssystem für CPS erfüllen?

Determinismus:

- Vorhersehbare Reaktion auf Ereignisse in einem festgelegten Zeitrahmen.

Low Latency:

- Schnelle Reaktionszeiten, insbesondere bei Steuerungen in Echtzeitumgebungen.

Welche Eigenschaften sollte ein Betriebssystem für CPS erfüllen?

Determinismus:

- Vorhersehbare Reaktion auf Ereignisse in einem festgelegten Zeitrahmen.

Low Latency:

- Schnelle Reaktionszeiten, insbesondere bei Steuerungen in Echtzeitumgebungen.

Zuverlässigkeit und Fehlertoleranz:

- Besonders bei sicherheitskritischen Systemen müssen Betriebssysteme stabil und ausfallsicher arbeiten.

-

Welche Eigenschaften sollte ein Betriebssystem für CPS erfüllen?

Determinismus:

- Vorhersehbare Reaktion auf Ereignisse in einem festgelegten Zeitrahmen.

Low Latency:

- Schnelle Reaktionszeiten, insbesondere bei Steuerungen in Echtzeitumgebungen.

Zuverlässigkeit und Fehlertoleranz:

- Besonders bei sicherheitskritischen Systemen müssen Betriebssysteme stabil und ausfallsicher arbeiten.

Multithreading und Parallelität:

- Unterstützung mehrerer paralleler Prozesse für effiziente Steuerung von Sensoren, Aktoren und Kommunikationsschnittstellen.

ROS – Robot Operating System

„The Robot Operating System (ROS) is a set of software libraries and tools that help you build robot applications. From drivers to state-of-the-art algorithms, and with powerful developer tools, ROS has what you need for your next robotics project. And it's all open source.“ Quelle: https://www.ros.org/

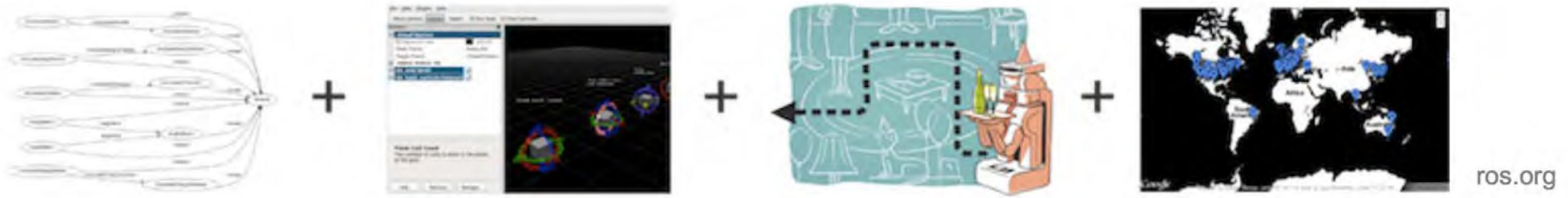
ROS – Robot Operating System

„The Robot Operating System (ROS) is a set of software libraries and tools that help you build robot applications. From drivers to state-of-the-art algorithms, and with powerful developer tools, ROS has what you need for your next robotics project. And it's all open source.“ Quelle: https://www.ros.org/

Es ersetzt nicht das Betriebssystem (bspw. Ubuntu)

„ROS is a meta-operating system which simplifies inter-process communication between elements of a robot‘s various sub-systems.“, Quelle: https://f1tenth-coursekit.readthedocs.io/en/latest/index.html

ROS – Robot Operating System

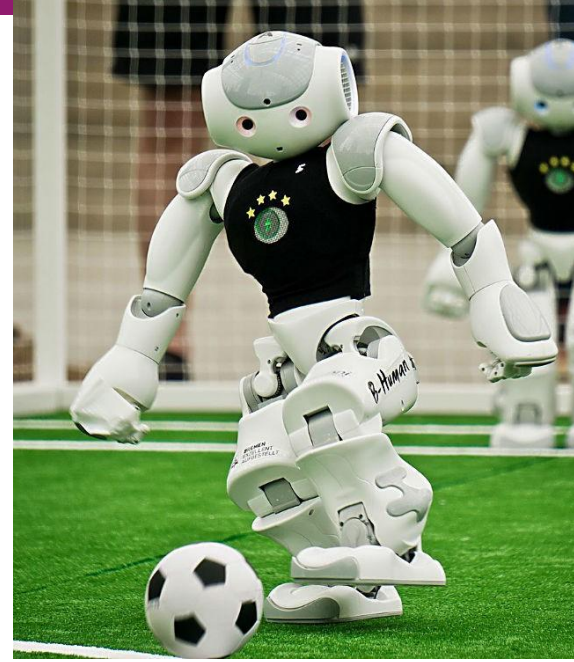


Plumbing	Tools	Capabilities	Ecosystem
▪ Process management	▪ Simulation	▪ Control	▪ Package organization
▪ Inter-process communication	▪ Visualization	▪ Planning	▪ Software distribution
▪ Device drivers	▪ Graphical user interface	▪ Perception	▪ Documentation
	▪ Data logging	▪ Mapping	▪ Tutorials
		▪ Manipulation	

ROS – Robot Operating System

Monolithic, Sequential

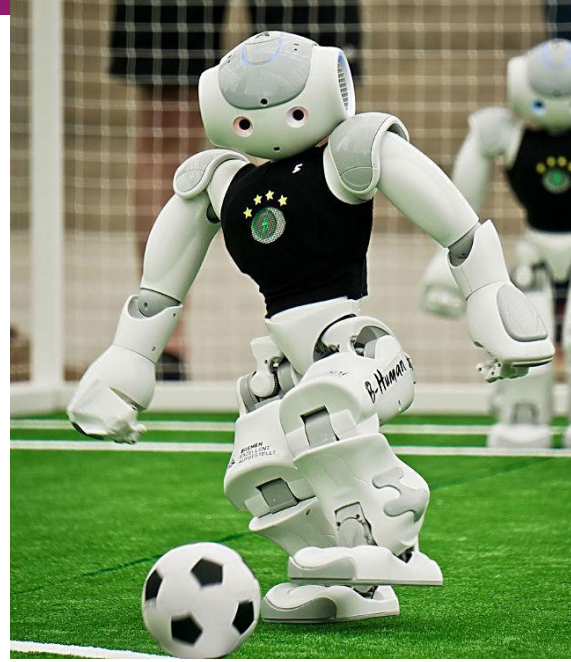
```
read_from_rgbd_camera(…)
detect_soccer_ball(…)
update_ball_tracker(…)
convert_rgbd_to_laser(…)
read_odometry(…)
update_localization(…)
update_motion_plan(…)
send_motion_commands(…)
```



medium.com

Quelle: M. Behl, F1tenth Autonomous Racing, UVA Link Lab, University of Virginia

ROS – Robot Operating System

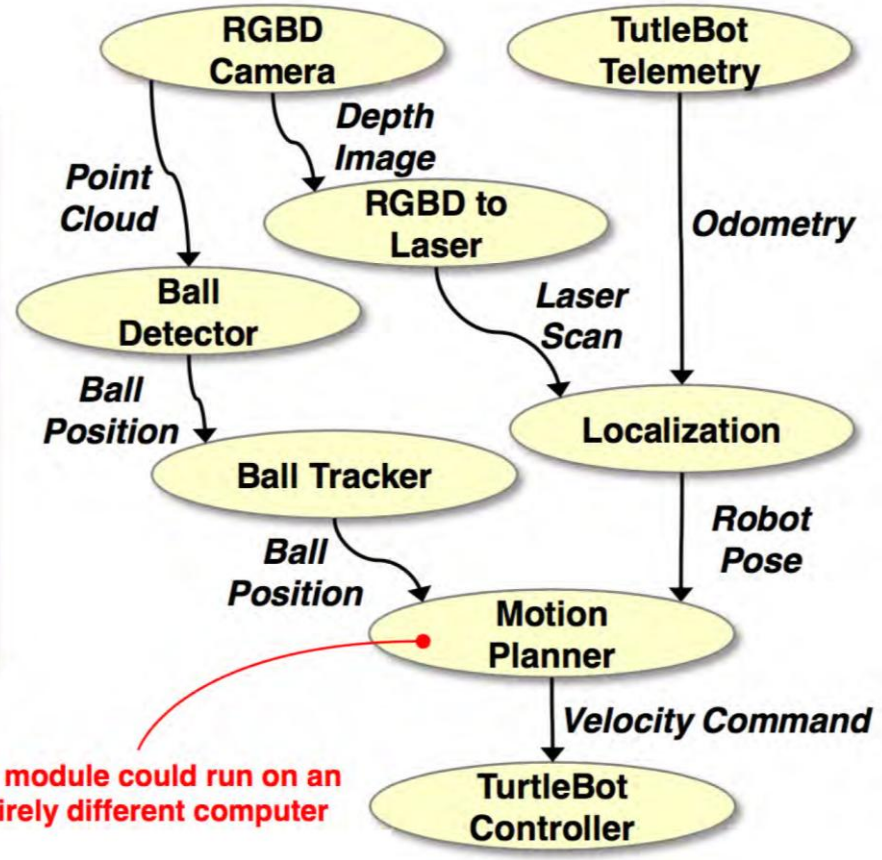


medium.com

Monolithic, Sequential

```
read_from_rgbd_camera(...)
detect_soccer_ball(...)
update_ball_tracker(...)
convert_rgbd_to_laser(...)
read_odometry(...)
update_localization(...)
update_motion_plan(...)
send_motion_commands(...)
```

Modular, Parallel



Quelle: M. Behl, F1tenth Autonomous Racing, UVA Link Lab, University of Virginia

ROS Philosophy

- **Peer to peer**
  Individual programs communicate over defined API (ROS *messages*, *services*, etc.).
- **Distributed**
  Programs can be run on multiple computers and communicate over the network.
- **Multi-lingual**
  ROS modules can be written in any language for which a client library exists (C++, Python, MATLAB, Java, etc.).
- **Light-weight**
  Stand-alone libraries are wrapped around with a thin ROS layer.
- **Free and open-source**
  Most ROS software is open-source and free to use.

ROS Distributionen

ROS Melodic

- Niryo NED2 Roboterarm
- Ubuntu 18.04





ROS Distributionen

ROS Melodic

- Niryo NED2 Roboterarm
- Ubuntu 18.04





ROS2 Humble

- Turtlebot
- Ubuntu 22.04









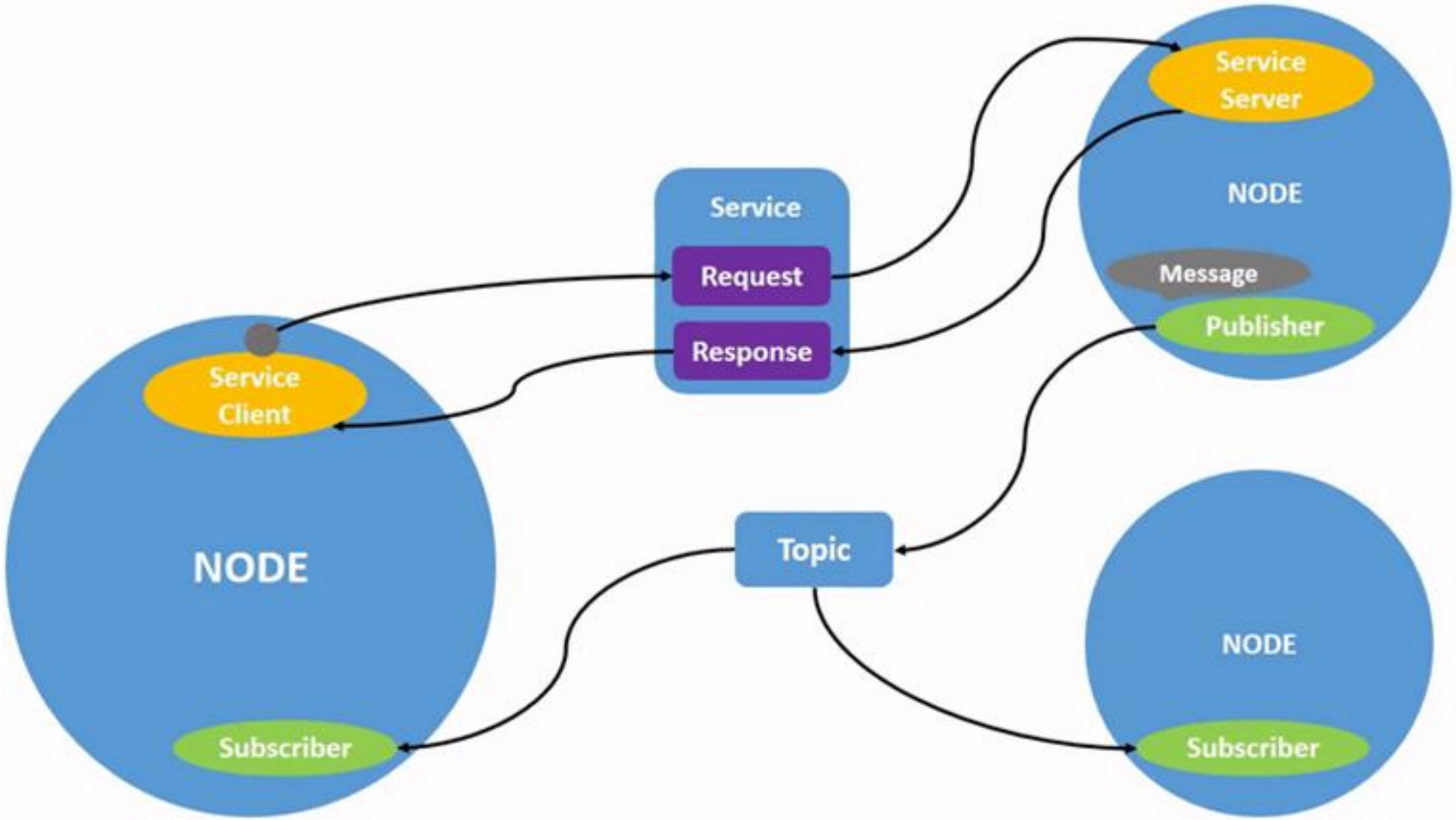


ROS2 Foxy

- Lite3

ROS graph

The ROS graph is a network of ROS 2 elements processing data together at one time. It encompasses all executables and the connections between them if you were to map them all out and visualize them.



Quelle: H. Zheng, F1tenth Autonomous Racing, Penn Engineering, University of Pennsylvania

https://docs.ros.org/en/foxy/Tutorials/Understanding-ROS2-Nodes.html

Nodes

Each node in ROS should be responsible for a single, module purpose (e.g. one node for controlling wheel motors, one node for controlling a laser range-finder, etc). Each node can send and receive data to other nodes via topics, services, actions, or parameters.

Quelle: https://docs.ros.org/en/foxy/Tutorials/Understanding-ROS2-Nodes.html

Nodes

Each node in ROS should be responsible for a single, module purpose (e.g. one node for controlling wheel motors, one node for controlling a laser range-finder, etc). Each node can send and receive data to other nodes via topics, services, actions, or parameters.

Related command line commands

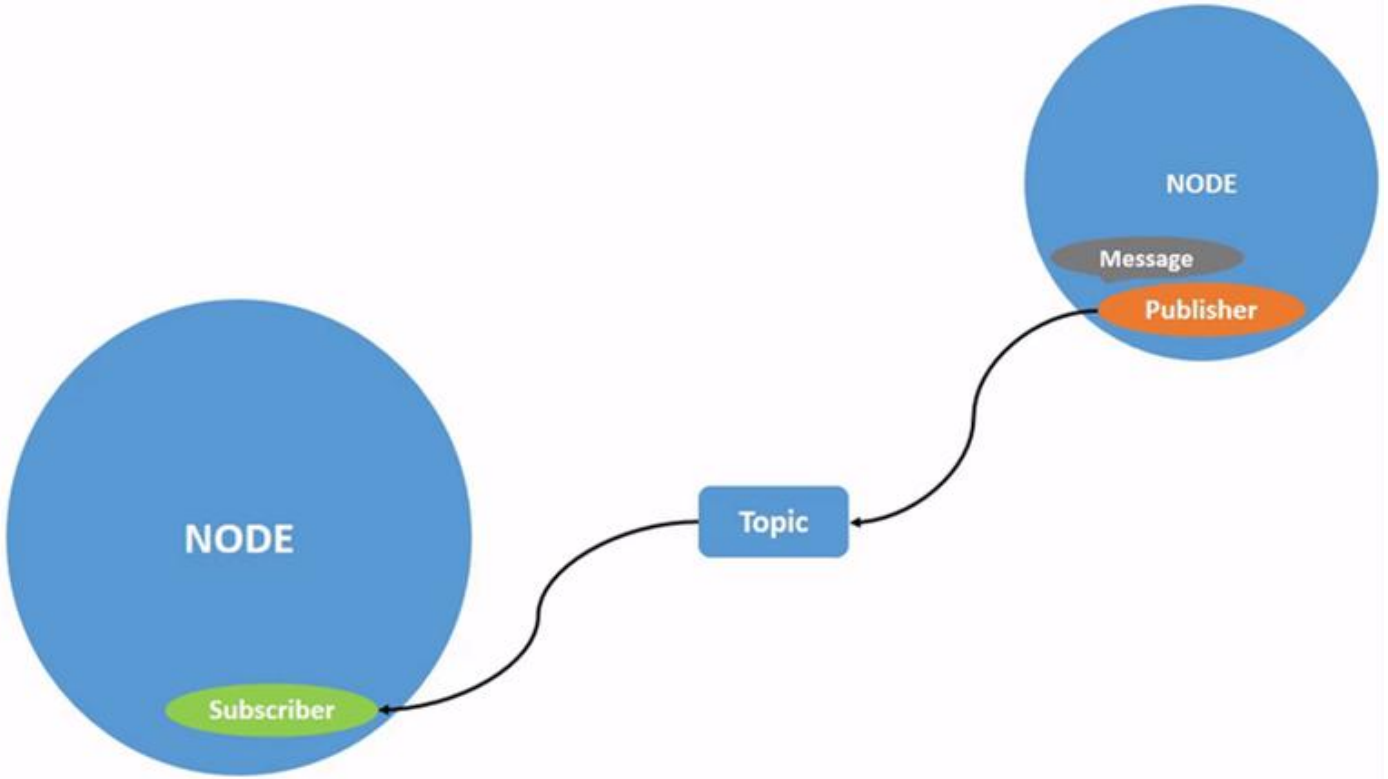
```
ros2 run <package_name> <executable_name>
ros2 node list
ros2 node info <node_name>
```

Quelle: https://docs.ros.org/en/foxy/Tutorials/Understanding-ROS2-Nodes.html

Topics

ROS 2 breaks complex systems down into many modular nodes. Topics are a vital element of the ROS graph that act as a bus for nodes to exchange messages.

Topics are for **streaming data**



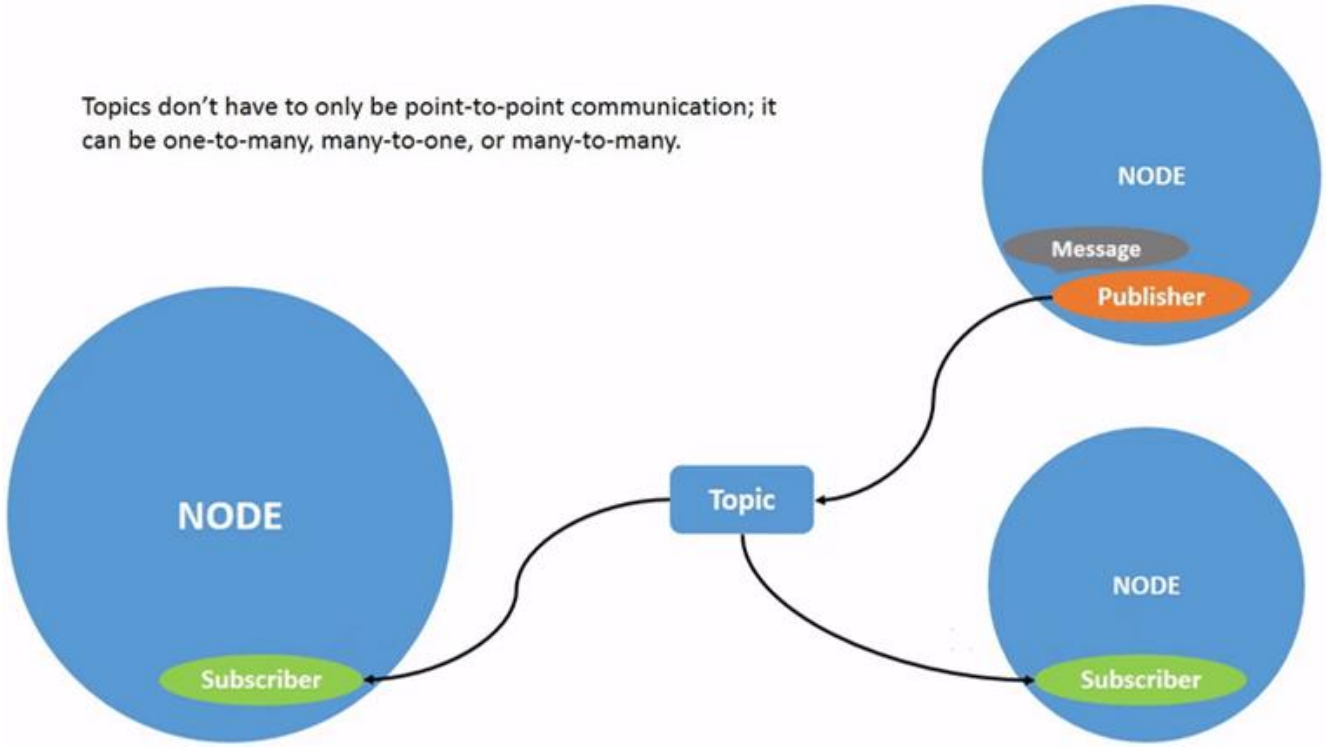
Quelle: H. Zheng, F1tenth Autonomous Racing, Penn Engineering, University of Pennsylvania

https://docs.ros.org/en/foxy/Tutorials/Topics/Understanding-ROS2-Topics.html

Topics

A node may publish data to any number of topics and simultaneously have subscriptions to any number of topics.

Topics are one of the main ways in which data is moved between nodes and therefore between different parts of the system.



Quelle: H. Zheng, F1tenth Autonomous Racing, Penn Engineering, University of Pennsylvania

https://docs.ros.org/en/foxy/Tutorials/Topics/Understanding-ROS2-Topics.html

Topics

Related command line commands

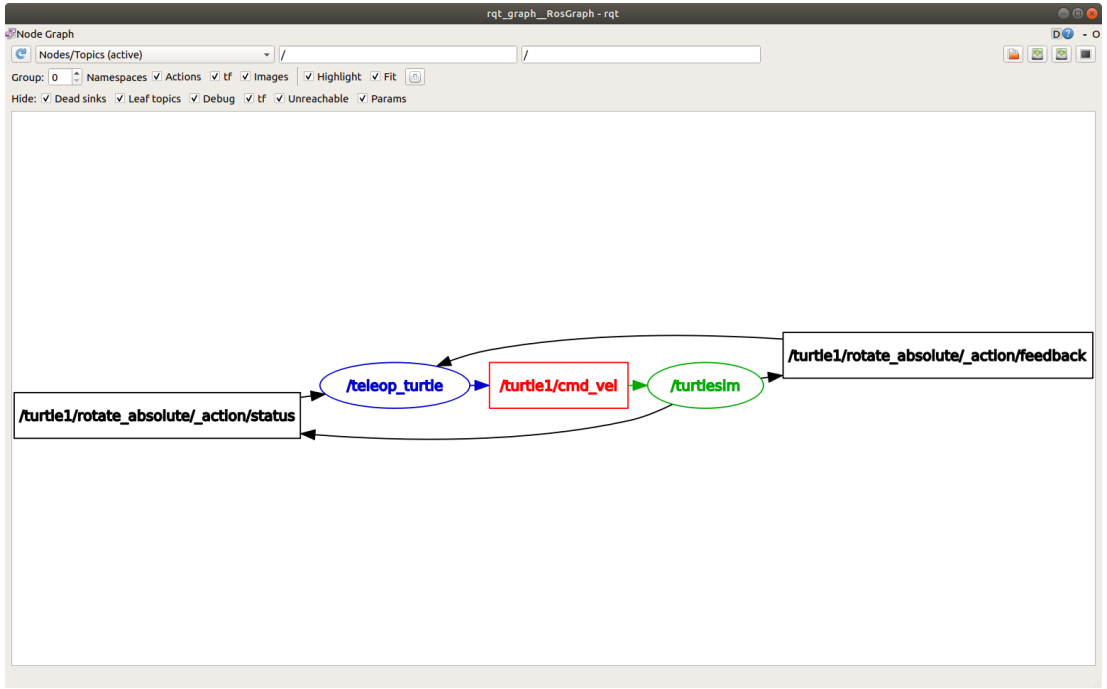
```
rqt_graph
ros2 topic list
ros2 topic list -t
ros2 topic echo <topic_name>
ros2 topic info <topic_name>
ros2 interface show <msg_type>
ros2 topic pub <topic_name> <msg_type> '<args>'
ros2 topic hz <topic_name>
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

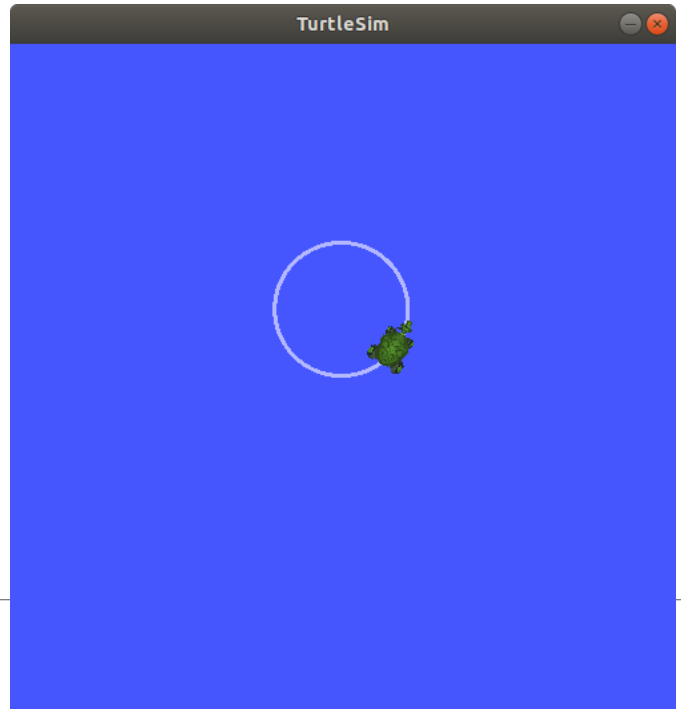
Quelle: H. Zheng, F1tenth Autonomous Racing, Penn Engineering, University of Pennsylvania

https://docs.ros.org/en/foxy/Tutorials/Topics/Understanding-ROS2-Topics.html

rqt_graph



- /teleop_turtle and /turtlesim are nodes
- /turtle1/cmd_vel is a topic



Quelle:
https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html

ROS Messages

Messages are the strongly-typed **data structure** for a topic.

Nodes communicate **messages** via **topics** in an asynchronous publish-subscribe model.

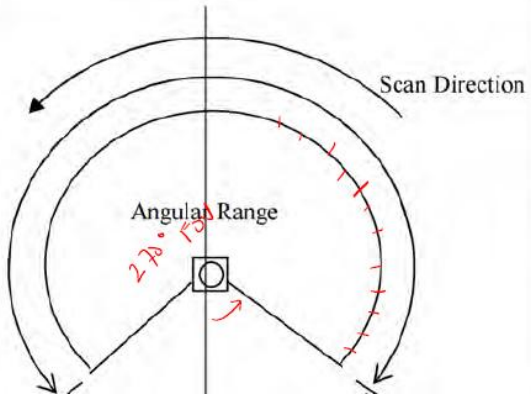


hokuyo_node

LaserScan [Message]

Scan [Topic]

Mapping Node



std_msgs/Header header
float32 angle_min
float32 angle_max
float32 angle_increment
float32 time_increment
float32 scan_time
float32 range_min
float32 range_max
float32[] ranges
float32[] intensities

Subscriber Node

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: M. Behl, F1tenth Autonomous Racing, UVA Link Lab, University of Virginia

Subscribing a Topic in Python

```
from sensor_msgs.msg import LaserScan

def __init__(self):
        …
        # Erstellen eines Abonnements für den LaserScan-Topic
         self.subscription = self.create_subscription(
             LaserScan, '/scan',
             self.listener_callback, self.qos_policy
         )

def listener_callback(self, msg):
        …
```

Working with data from a Laserscan

```
def listener_callback(self, msg):
    angle_min = msg.angle_min
    # Berechnung des Index für die Front des Roboters
    index_front = round((-1.5708 - angle_min) / msg.angle_increment)  # -1.5708 rad entspricht -90 Grad

    # Überprüfen, ob ein Hindernis im 50-Grad-Bereich vor dem Roboter ist
    index_start = max(0, index_front - 50)
    index_end = min(len(msg.ranges) - 1, index_front + 50)
    front_ranges = msg.ranges[index_start:index_end+1]
    front_distance = min([x for x in front_ranges if x != float('inf')])
```

Publish to a Topic in Python

```
from geometry_msgs.msg import Twist

def __init__(self):
        …
        # Erstellen eines Publishers für den cmd_vel-Topic
         self.publisher = self.create_publisher(
             Twist, '/cmd_vel‘, 10  # Standard-QoS
         )

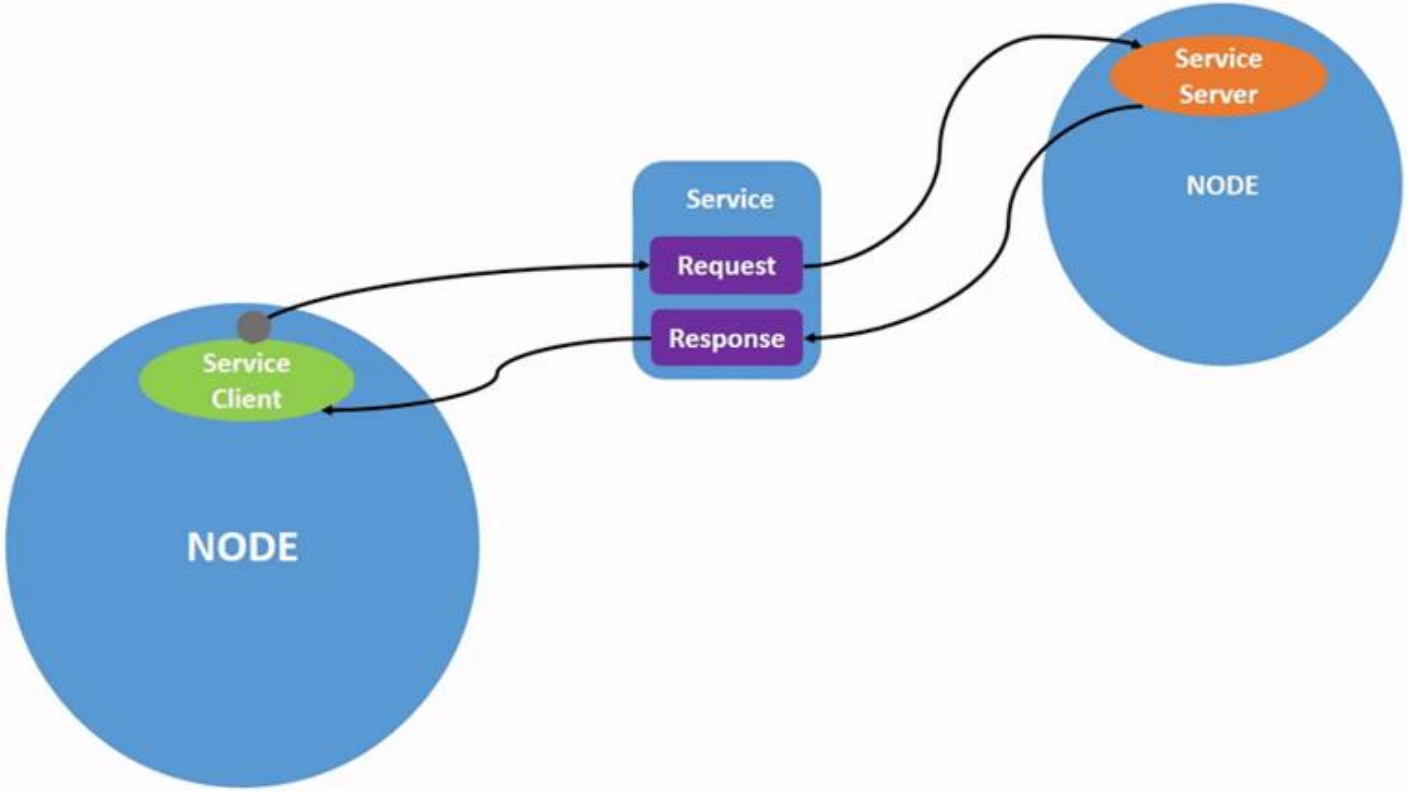
…
        self.publisher.publish(self.twist)
```

```
self.twist = Twist()

# Drehe Turtlebot um 45 Grad
self.twist.linear.x = 0.0
# 45 Grad in Rad
self.twist.angular.z = math.pi / 4 * self.turn_direction
```

Services

- Services are based on a call-and-response model, versus topics' publisher-subscriber model.
-
-
  -



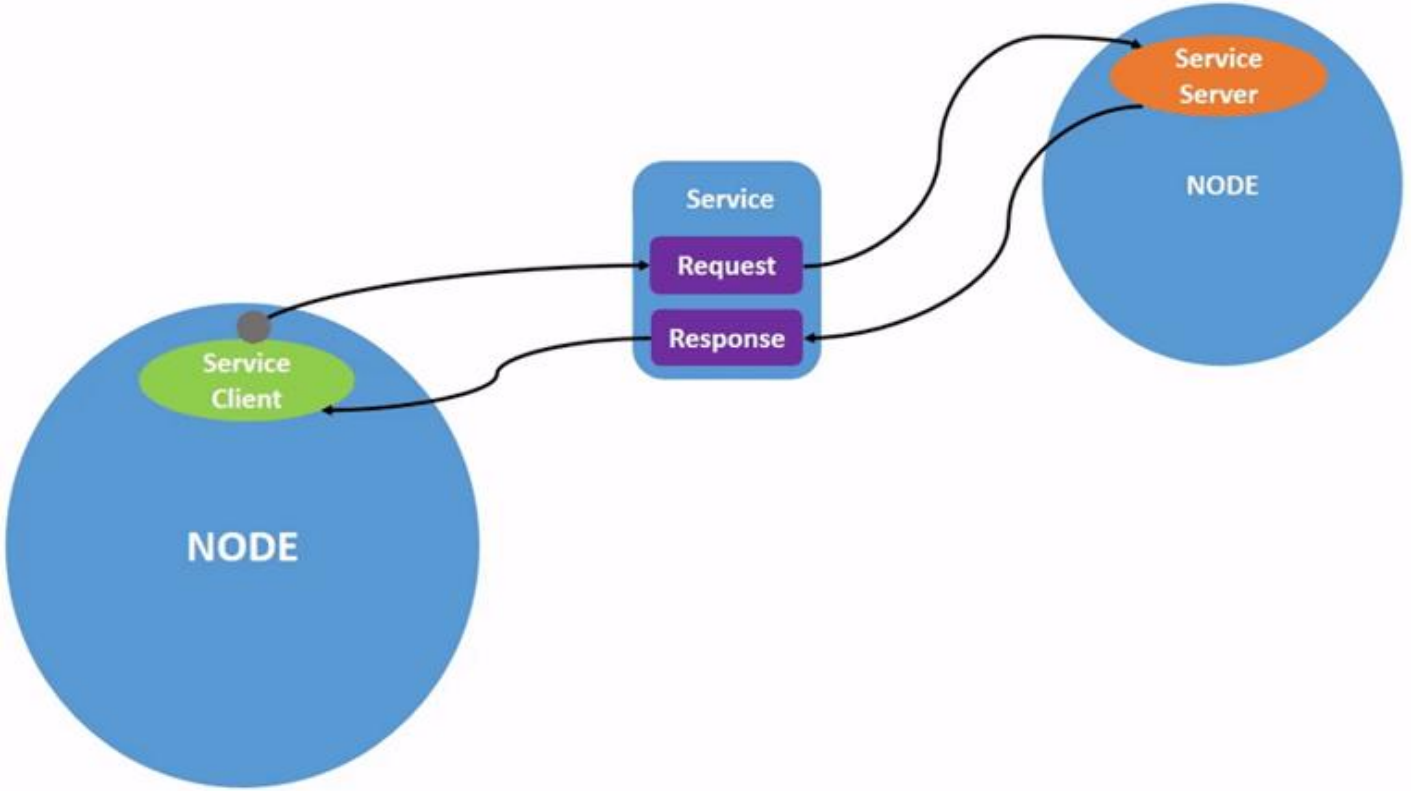
Quelle: H. Zheng, F1tenth Autonomous Racing, Penn Engineering, University of Pennsylvania

https://docs.ros.org/en/foxy/Tutorials/Services/Understanding-ROS2-Services.html

Services

- Services are based on a call-and-response model, versus topics' publisher-subscriber model.
- While topics allow nodes to subscribe to data streams and get continual updates, services only provide data when they are specifically called by a client.

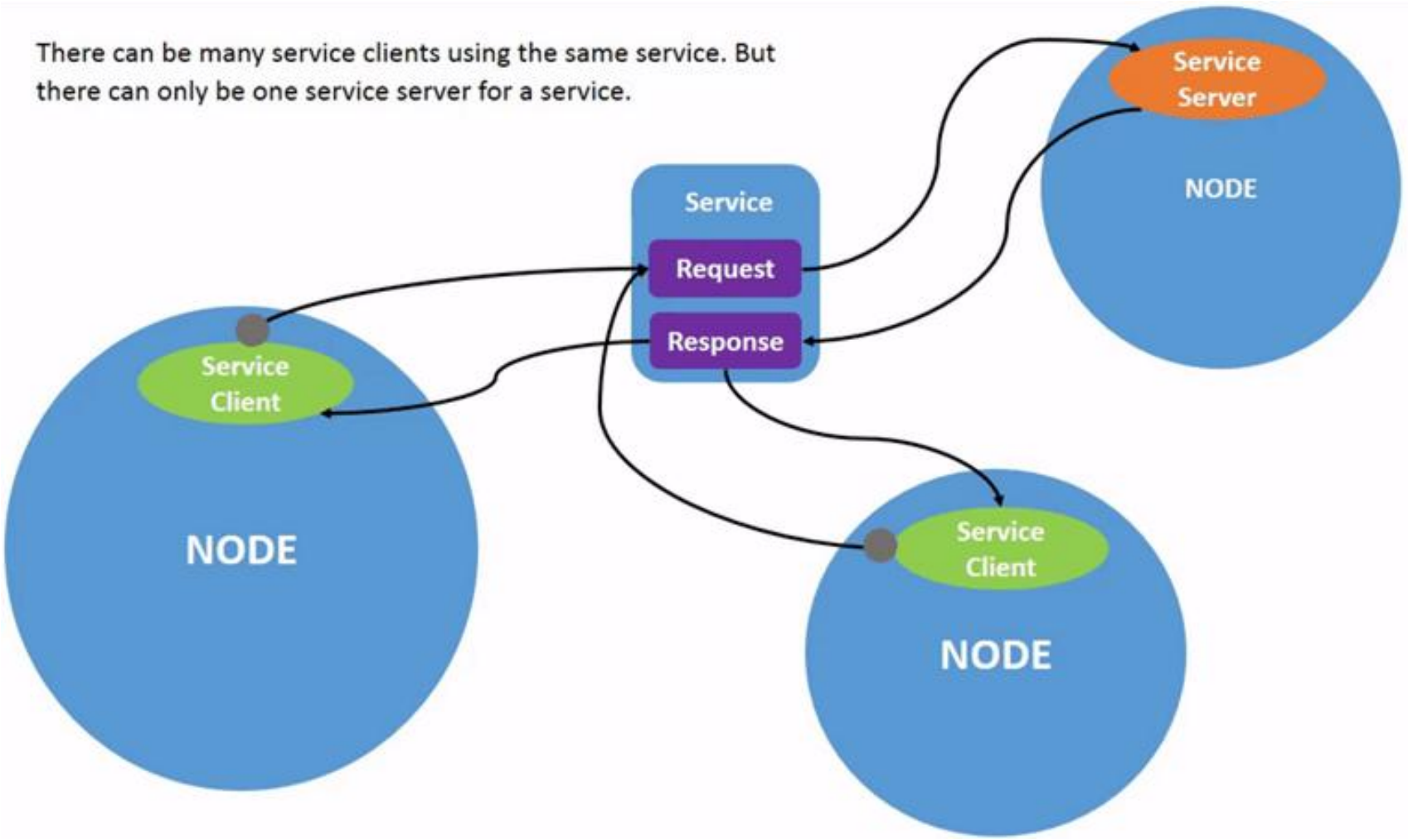
- Example:
  - get a value/configuration of the server node



Quelle: H. Zheng, F1tenth Autonomous Racing, Penn Engineering, University of Pennsylvania

https://docs.ros.org/en/foxy/Tutorials/Services/Understanding-ROS2-Services.html

Services



Quelle: H. Zheng, F1tenth Autonomous Racing, Penn Engineering, University of Pennsylvania

https://docs.ros.org/en/foxy/Tutorials/Services/Understanding-ROS2-Services.html

Services

Related command line commands

```
ros2 service list
ros2 service type <service_name>
ros2 service list -t
ros2 service find <type_name>
ros2 interface show <type_name>.srv
ros2 service call <service_name> <service_type> <arguments>
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Quelle: H. Zheng, F1tenth Autonomous Racing, Penn Engineering, University of Pennsylvania
https://docs.ros.org/en/foxy/Tutorials/Services/Understanding-ROS2-Services.html

Erfüllt ROS 2 die Anforderungen an das Betriebssystem für CPS?

Determinismus und Multithreading:

- ROS 2 verwendet einen **Executor**, der die Steuerung der Reihenfolge und des Zeitpunkts der Verarbeitung von Nachrichten ermöglicht.
-

- https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Executors.html

Erfüllt ROS 2 die Anforderungen an das Betriebssystem für CPS?

Determinismus und Multithreading:

- ROS 2 verwendet einen **Executor**, der die Steuerung der Reihenfolge und des Zeitpunkts der Verarbeitung von Nachrichten ermöglicht.
-

- https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Executors.html

```
robo_control = RoboControl()
executor = MultiThreadedExecutor()
executor.add_node(robo_control)
executor.spin()
```

Erfüllt ROS 2 die Anforderungen an das Betriebssystem für CPS?

Determinismus und Multithreading:

- ROS 2 verwendet einen **Executor**, der die Steuerung der Reihenfolge und des Zeitpunkts der Verarbeitung von Nachrichten ermöglicht.
- ROS 2 unterstützt Mehrkanal- und Multi-Thread-Kommunikation, um parallele und priorisierte Aufgaben zu handhaben.

- https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Executors.html

```
robo_control = RoboControl()
executor = MultiThreadedExecutor()
executor.add_node(robo_control)
executor.spin()
```

Erfüllt ROS 2 die Anforderungen an das Betriebssystem für CPS?

Determinismus und Multithreading:

- ROS 2 verwendet einen **Executor**, der die Steuerung der Reihenfolge und des Zeitpunkts der Verarbeitung von Nachrichten ermöglicht.
- ROS 2 unterstützt Mehrkanal- und Multi-Thread-Kommunikation, um parallele und priorisierte Aufgaben zu handhaben.

Low Latency:

- Die asynchrone Kommunikation und Middleware (DDS) von ROS 2 ermöglichen eine Latenzreduktion, aber die genaue Latenz hängt von dem unterliegenden Betriebssystem und Hardwarekonfiguration ab.

Erfüllt ROS 2 die Anforderungen an das Betriebssystem für CPS?

Determinismus und Multithreading:

- ROS 2 verwendet einen **Executor**, der die Steuerung der Reihenfolge und des Zeitpunkts der Verarbeitung von Nachrichten ermöglicht.
- ROS 2 unterstützt Mehrkanal- und Multi-Thread-Kommunikation, um parallele und priorisierte Aufgaben zu handhaben.

Low Latency:

- Die asynchrone Kommunikation und Middleware (DDS) von ROS 2 ermöglichen eine Latenzreduktion, aber die genaue Latenz hängt von dem unterliegenden Betriebssystem und Hardwarekonfiguration ab.

Zuverlässigkeit und Fehlertoleranz:

- ROS 2 bietet integrierte Fehlertoleranzmechanismen wie die Wiederholung fehlgeschlagener Nachrichtenübertragungen und robuste Kommunikationsprotokolle.

Einführung in DDS (Data Distribution Service) in ROS 2

Was ist DDS?

- DDS ist ein Standard für Middleware-Kommunikation, der Peer-to-Peer-Datenverteilung in Echtzeitsystemen ermöglicht.
- DDS ist auf hohe Skalierbarkeit und Flexibilität ausgelegt, wodurch es sich ideal für verteilte Systeme wie CPS eignet.
-
-
-

Einführung in DDS (Data Distribution Service) in ROS 2

Was ist DDS?

- DDS ist ein Standard für Middleware-Kommunikation, der Peer-to-Peer-Datenverteilung in Echtzeitsystemen ermöglicht.
- DDS ist auf hohe Skalierbarkeit und Flexibilität ausgelegt, wodurch es sich ideal für verteilte Systeme wie CPS eignet.

DDS in ROS 2:

- ROS 2 verwendet DDS als seine Standard-Middleware.
- DDS kümmert sich um die Datenverteilung zwischen den Nodes in ROS 2, unabhängig davon, ob die Nodes auf derselben Maschine oder über ein Netzwerk verteilt laufen.
- DDS stellt sicher, dass Nachrichten verlustfrei und deterministisch übertragen werden, was für Echtzeitanwendungen entscheidend ist.

Vorteile von DDS für Echtzeitkommunikation in ROS 2

QoS (Quality of Service) in DDS:

- DDS ermöglicht die Feinanpassung der Kommunikationsanforderungen durch (QoS)-Profile.
- Diese Profile definieren Kommunikationsparameter wie Zuverlässigkeit, Latenz, Deadline und Bandbreitennutzung.

Vorteile von DDS für Echtzeitkommunikation in ROS 2

QoS (Quality of Service) in DDS:

- DDS ermöglicht die Feinanpassung der Kommunikationsanforderungen durch (QoS)-Profile.
- Diese Profile definieren Kommunikationsparameter wie Zuverlässigkeit, Latenz, Deadline und Bandbreitennutzung.

```
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy, Deadline

qos_profile = QoSProfile(history=HistoryPolicy.KEEP_LAST,
    depth=10,  # Zahl von Nachrichten, die gespeichert werden (Bandbreitennutzung)
    reliability=ReliabilityPolicy.RELIABLE,  # Zuverlässigkeit der Übertragung
    deadline=Deadline(seconds=0.1)  # Deadline von 100 ms für das Senden der Nachricht
                                    (Latenz & Deadline)
)

self.publisher = self.create_publisher(String, 'example_topic', qos_profile)
```

Vorteile von DDS für Echtzeitkommunikation in ROS 2

QoS (Quality of Service) in DDS:

- DDS ermöglicht die Feinanpassung der Kommunikationsanforderungen durch (QoS)-Profile.
- In Echtzeit-Anwendungen können spezifische QoS-Einstellungen verwendet werden, um Nachrichtenverlust zu vermeiden oder priorisierte Datenübertragung sicherzustellen.

Vorteile von DDS für Echtzeitkommunikation in ROS 2

QoS (Quality of Service) in DDS:

- DDS ermöglicht die Feinanpassung der Kommunikationsanforderungen durch (QoS)-Profile.
- In Echtzeit-Anwendungen können spezifische QoS-Einstellungen verwendet werden, um Nachrichtenverlust zu vermeiden oder priorisierte Datenübertragung sicherzustellen.

```
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy

best_effort_qos_policy = QoSProfile(history=HistoryPolicy.KEEP_LAST, depth=1,
                                    reliability=ReliabilityPolicy.BEST_EFFORT)

self.movement_publisher = self.create_publisher(Twist, "/cmd_vel",
                                    best_effort_qos_policy)
```

Vorteile von DDS für Echtzeitkommunikation in ROS 2

Echtzeitkommunikation mit DDS in ROS 2:

- Determinismus: DDS stellt sicher, dass Nachrichten rechtzeitig und in vorhersehbarer Weise übermittelt werden.
- Skalierbarkeit: DDS unterstützt Systeme von kleinen eingebetteten Geräten bis hin zu großen verteilten Netzwerken.
- Zuverlässigkeit: DDS garantiert, dass kritische Daten bei Bedarf mehrfach gesendet oder die Übertragung bei Fehlern wiederholt wird.

Vorteile von DDS für Echtzeitkommunikation in ROS 2

Echtzeitkommunikation mit DDS in ROS 2:

- Determinismus: DDS stellt sicher, dass Nachrichten rechtzeitig und in vorhersehbarer Weise übermittelt werden.
- Skalierbarkeit: DDS unterstützt Systeme von kleinen eingebetteten Geräten bis hin zu großen verteilten Netzwerken.
- Zuverlässigkeit: DDS garantiert, dass kritische Daten bei Bedarf mehrfach gesendet oder die Übertragung bei Fehlern wiederholt wird.

```
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy
reliable_qos_policy = QoSProfile(history=HistoryPolicy.KEEP_LAST,
    depth=10,  # Puffergröße, um mehrere Nachrichten zu speichern
    reliability=ReliabilityPolicy.RELIABLE)
self.movement_publisher = self.create_publisher(Twist, "/cmd_vel", reliable_qos_policy)
```

Parameters

A parameter is a configuration value of a node. You can think of parameters as node settings. A node can store parameters as integers, floats, booleans, strings and lists. In ROS 2, each node maintains its own parameters. All parameters are dynamically reconfigurable, and built off of ROS 2 services.

Can start your node with different settings each time, without having to change your Python code.

Quelle: H. Zheng, F1tenth Autonomous Racing, Penn Engineering, University of Pennsylvania https://docs.ros.org/en/foxy/Tutorials/Parameters/Understanding-ROS2-Parameters.html

Parameters

A parameter is a configuration value of a node. You can think of parameters as node settings. A node can store parameters as integers, floats, booleans, strings and lists. In ROS 2, each node maintains its own parameters. All parameters are dynamically reconfigurable, and built off of ROS 2 services.

Can start your node with different settings each time, without having to change your Python code.

Related command line commands

ros2 param list

ros2 param get <node_name> <parameter_name>

ros2 param set <node_name> <parameter_name> <value>

Examples: speed of turtlebot (full speed, slow speed, …); minimum distance to an object, …

Quelle: H. Zheng, F1tenth Autonomous Racing, Penn Engineering, University of Pennsylvania https://docs.ros.org/en/foxy/Tutorials/Parameters/Understanding-ROS2-Parameters.html

Parameters

Parameters could also be set in either a launch file or a yaml file.

Useful to have all parameters in one place while tuning.

Set parameters for multiple nodes in one file.

For a full length tutorial:
https://roboticsbackend.com/ros2-yaml-params/

```
node_1:
  ros__parameters:
    some_text: "abc"

node_2:
  ros__parameters:
    int_number: 27
    float_param: 45.2

node_3:
  ros__parameters:
    int_number: 45
```


Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: H. Zheng, F1tenth Autonomous Racing, Penn Engineering, University of Pennsylvania

Parameters

Getting ROS parameters programmatically:

In a node, declare a parameter first

- self.declare_parameter('my_param')

Then getting a parameter

- self.get_parameter('my_param')

Parameters

Getting ROS parameters programmatically:

In a node, declare a parameter first

- self.declare_parameter('my_param')

Then getting a parameter

- self.get_parameter('my_param')

You could also get multiple parameters at once

Similar in C++

For full tutorials on parameters see:

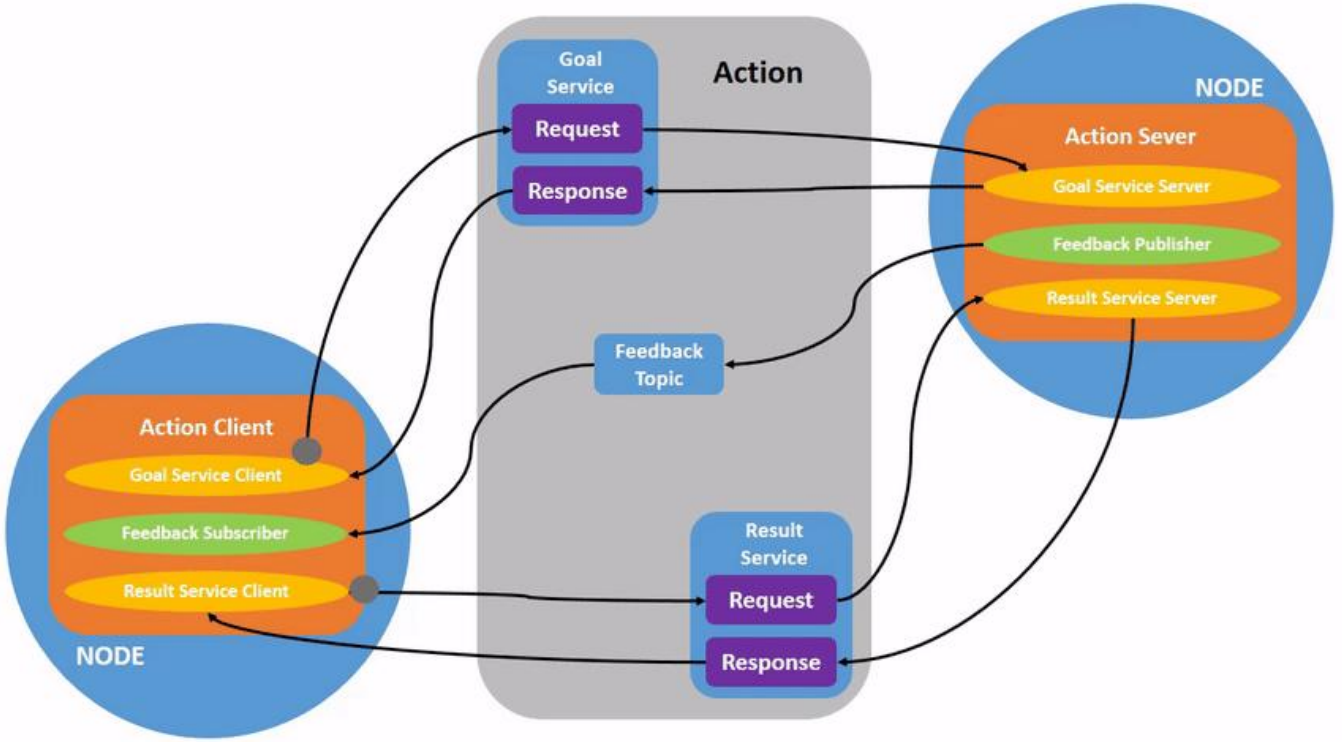
- https://roboticsbackend.com/rclpy-params-tutorial-get-set-ros2-params-with-python/
- https://roboticsbackend.com/rclcpp-params-tutorial-get-set-ros2-params-with-cpp/

Actions

Action: „Fahre in die Küche“

Actions are one of the communication types in ROS 2 and are intended for long running tasks. They consist of three parts: a goal, feedback, and a result.

Actions are built on topics and services. Their functionality is similar to services, except actions are preemptable (you can cancel them while executing). They also provide steady feedback, as opposed to services which return a single response.



Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

Quelle: H. Zheng, F1tenth Autonomous Racing, Penn Engineering, University of Pennsylvania

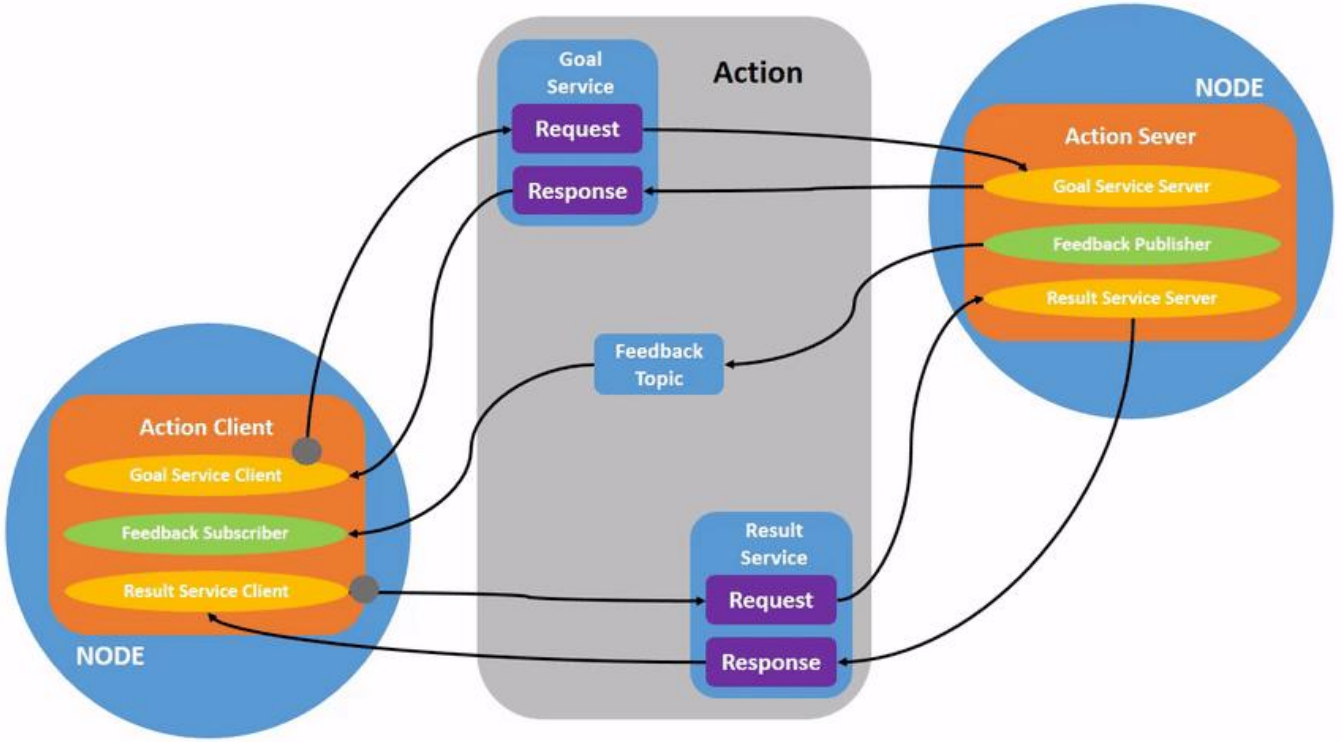
https://docs.ros.org/en/foxy/Tutorials/Understanding-ROS2-Actions.html

Actions

Action: „Fahre in die Küche“

Actions are one of the communication types in ROS 2 and are intended for long running tasks. They consist of three parts: a goal, feedback, and a result.

Actions are built on topics and services. Their functionality is similar to services, except actions are preemptable (you can cancel them while executing). They also provide steady feedback, as opposed to services which return a single response.



Actions use a client-server model, similar to the publisher-subscriber model (described in the topics tutorial).
An “action client” node sends a goal to an “action server” node that acknowledges the goal and returns a stream of feedback and a result.

Quelle: H. Zheng, F1tenth Autonomous Racing, Penn Engineering, University of Pennsylvania

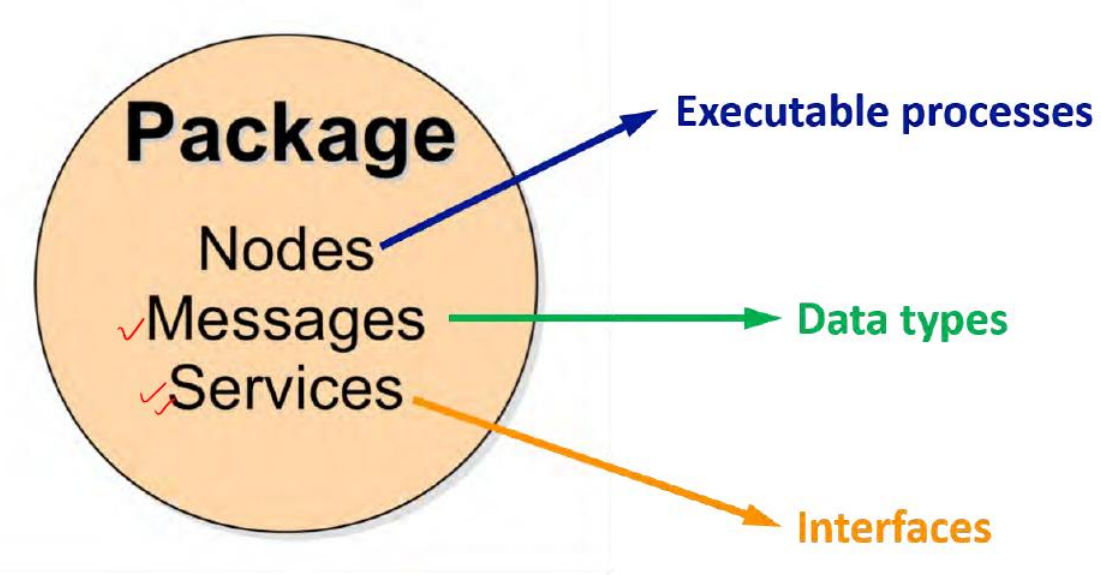
https://docs.ros.org/en/foxy/Tutorials/Understanding-ROS2-Actions.html

ROS graph - summary

- Modules are implemented in nodes
- Nodes can communicate via
  - Topics (streaming data)
  - Services
  - Actions (long running tasks)

ROS 2 Packages

A package can be considered a container for your ROS 2 code. If you want to be able to install your code or share it with others, then you'll need it organized in a package. With packages, you can release your ROS 2 work and allow others to build and use it easily.



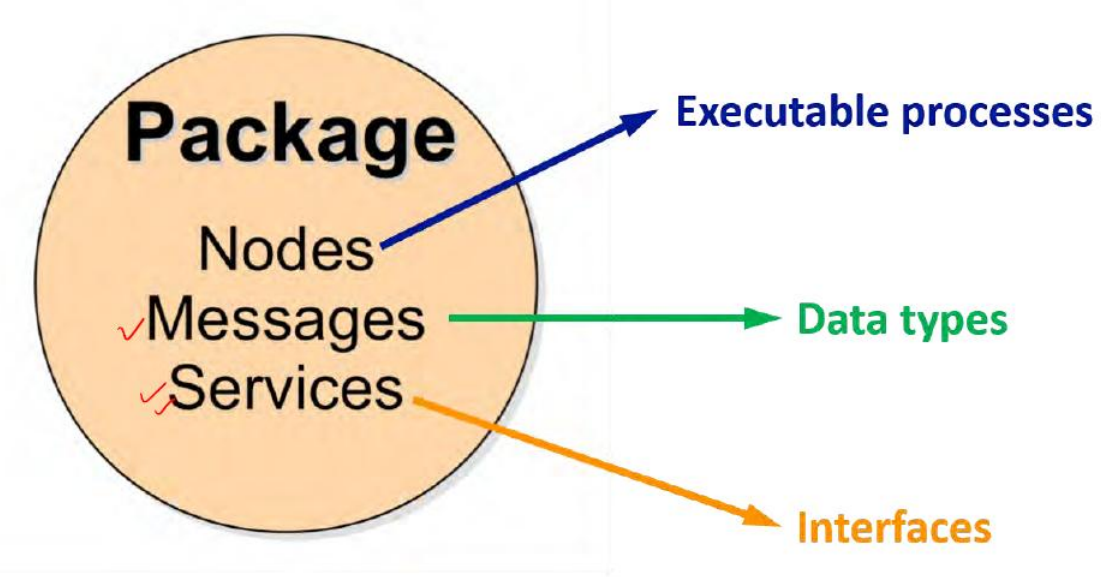
Bildquelle: M. Behl, F1tenth Autonomous Racing, UVA Link Lab, University of Virginia

ROS 2 Packages

A package can be considered a container for your ROS 2 code. If you want to be able to install your code or share it with others, then you'll need it organized in a package. With packages, you can release your ROS 2 work and allow others to build and use it easily.

Package creation in ROS 2 uses ament as its build system and colcon as its build tool.
You can create a package using either CMake or Python, which are officially supported, though other build types do exist.

A **package** contains one or more **nodes**.



Bildquelle: M. Behl, F1tenth Autonomous Racing, UVA Link Lab, University of Virginia

Workspace

A workspace is a directory containing ROS 2 packages. Before using ROS 2, it’s necessary to source your ROS 2 installation workspace in the terminal you plan to work in. This makes ROS 2’s packages available for you to use in that terminal.

Workspace

A workspace is a directory containing ROS 2 packages. Before using ROS 2, it’s necessary to source your ROS 2 installation workspace in the terminal you plan to work in. This makes ROS 2’s packages available for you to use in that terminal.

```
# Replace ".bash" with your shell if you're not using bash
# Possible values are: setup.bash, setup.sh, setup.zsh
source /opt/ros/humble/setup.bash
```


Quelle: H. Zheng, F1tenth Autonomous Racing, Penn Engineering, University of Pennsylvania

Workspace

Defines context for the current workspace

Creating a new workspace

- $ mkdir -p <your_workspace>/src
- Then you can put your desired ROS2 packages inside src

ROS 2 Packages

Creating a package:

- $ cd <your_workspace>/src
- (Python packages): $ ros2 pkg create --build-type ament_python <package_name>
- (CMake packages): $ ros2 pkg create --build-type ament_cmake <package_name>

ROS 2 Packages

Python and CMake packages each have their own minimum requirements.

CMake packages:

- package.xml: file containing meta info about the package
- CMakeLists.txt: file describing how to build the code within the package

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

ROS 2 Packages

Python and CMake packages each have their own minimum requirements.

CMake packages:

- package.xml: file containing meta info about the package
- CMakeLists.txt: file describing how to build the code within the package

Python packages:

- package.xml: file containing meta info about the package
- setup.py: contains instructions for how to install the package
- setup.cfg: required when a package has executables, so ros2 run can find them
- /<package_name>: a directory with the same name as your package, used by ROS 2 tools to find your package, contains __init__.py

ROS 2 Packages

The simplest package may have a file structure that looks like:

CMake:

- src/my_package/
  CMakeLists.txt
  package.xml

ROS 2 Packages

The simplest package may have a file structure that looks like:

CMake:

- src/my_package/
  - CMakeLists.txt
  - package.xml

Python:

- src/my_package/
  - setup.py
  - package.xml
  - my_package
    - __init__.py
    - ….py

ROS 2 Packages

A single workspace can contain as many packages as you want, each in their own folder. You can also have packages of different build types in one workspace (CMake, Python, etc.). You cannot have nested packages.

ROS 2 Packages

```
workspace_folder/
    src/
      package_1/
          CMakeLists.txt
          package.xml

      package_2/
          setup.py
          package.xml
          package_2
      ...
      package_n/
          CMakeLists.txt
          package.xml
```

A workspace with multiple packages might look like this:

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik

H. Zheng, F1tenth Autonomous Racing, Penn Engineering, University of Pennsylvania

Workspace

Resolving dependencies before building the workspace

- $ cd <your_workspace>
- $ rosdep install -i --from-path src --rosdistro galactic –y

- rosdep installs all packages needed by your packages inside the workspace

Workspace

Build the workspace with colcon:

- From the root of your workspace: $ colcon build
- Useful arguments when building:
  - --packages-up-to: builds the package you want, plus all its dependencies, but not the whole workspace. This will save some time if you don't need all the packages in the workspace.
  - --symlink-install: saves you from having to rebuild every time you tweak Python scripts
  - --event-handlers console_direct+: shows console output while building (can otherwise be found in the log directory)

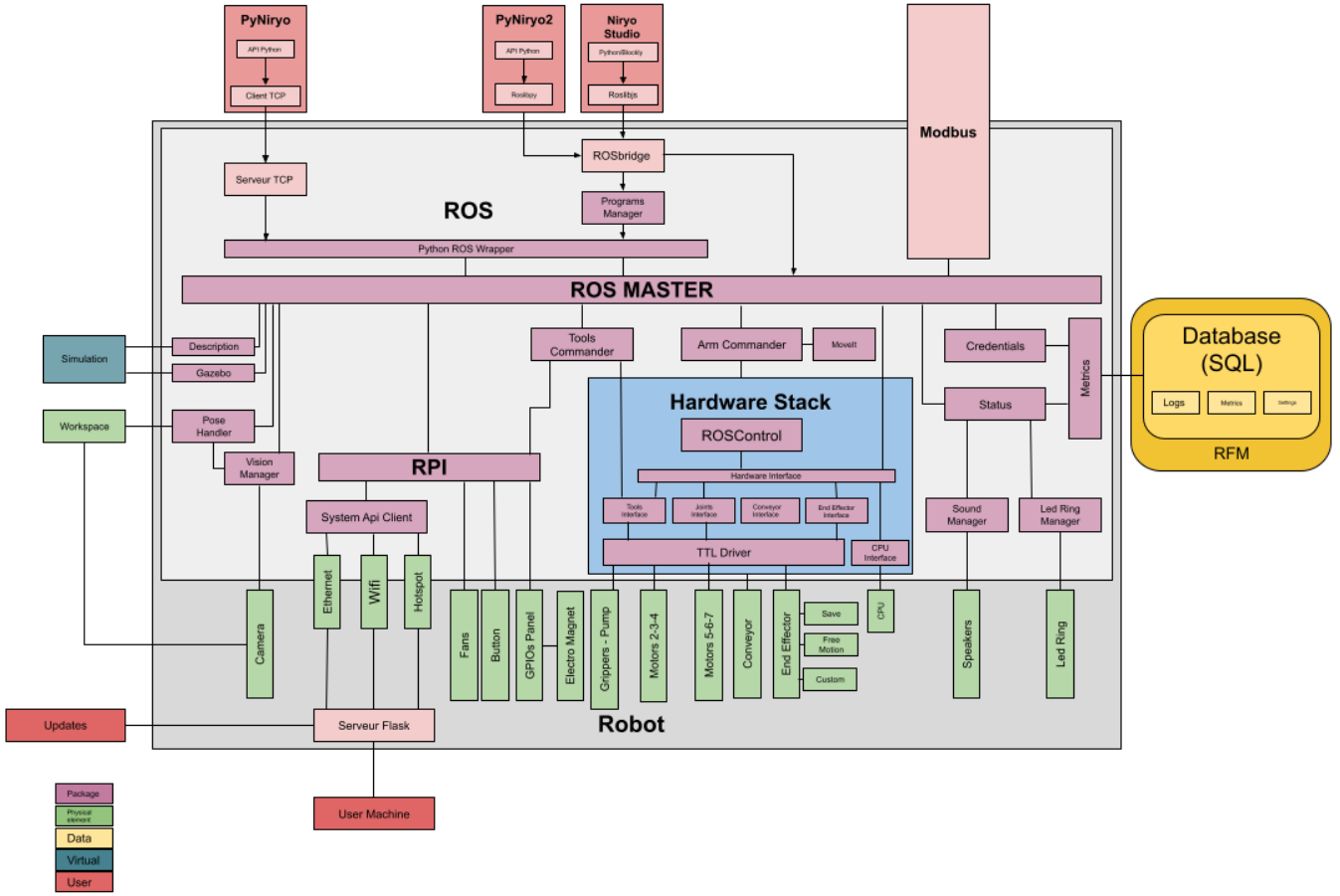
Once build finishes, you'll see

- build, install, log, src

directories in your workspace. The install directory is where your workspace's setup files are.

ROS Stack von Niryo NED 2

- ROS und nicht ROS 2



Simulation von Robotern in Gazebo

Was ist Gazebo?

- Gazebo ist ein Open-Source-Simulator für Robotik, der es ermöglicht, Roboter und deren Verhalten in komplexen, realitätsnahen 3D-Umgebungen zu simulieren.

Simulation von Robotern in Gazebo

Was ist Gazebo?

- Gazebo ist ein Open-Source-Simulator für Robotik, der es ermöglicht, Roboter und deren Verhalten in komplexen, realitätsnahen 3D-Umgebungen zu simulieren.

Warum Simulation? Die Simulation mit Gazebo bietet gegenüber realen Experimenten zahlreiche Vorteile:

- Kosteneffizienz: Virtuelle Tests sind kostengünstiger als der Bau und Betrieb realer Prototypen.
- Sicherheit: In der Simulation können risikoreiche Szenarien gefahrlos getestet werden.
- Reproduzierbarkeit: Simulationsläufe können beliebig oft wiederholt werden, um die Ergebnisse zu validieren.
- Skalierbarkeit: Virtuelle Umgebungen können leicht angepasst und erweitert werden.

Modellierung von Robotern und Umgebungen in Gazebo

Robotermodelle:

- Gazebo bietet eine Bibliothek von vorgefertigten Robotermodellen (z.B. TurtleBot). Eigene Modelle können mit Tools wie URDF (Unified Robot Description Format) erstellt werden.

Modellierung von Robotern und Umgebungen in Gazebo

Robotermodelle:

- Gazebo bietet eine Bibliothek von vorgefertigten Robotermodellen (z.B. TurtleBot). Eigene Modelle können mit Tools wie URDF (Unified Robot Description Format) erstellt werden.

Sensoren und Aktoren:

- Sensoren (z.B. LiDAR, Kameras) und Aktoren (z.B. Motoren, Greifer) können zu den Robotermodellen hinzugefügt werden. Gazebo simuliert das Verhalten dieser Komponenten realitätsnah.

Modellierung von Robotern und Umgebungen in Gazebo

Robotermodelle:

- Gazebo bietet eine Bibliothek von vorgefertigten Robotermodellen (z.B. TurtleBot). Eigene Modelle können mit Tools wie URDF (Unified Robot Description Format) erstellt werden.

Sensoren und Aktoren:

- Sensoren (z.B. LiDAR, Kameras) und Aktoren (z.B. Motoren, Greifer) können zu den Robotermodellen hinzugefügt werden. Gazebo simuliert das Verhalten dieser Komponenten realitätsnah.

Umgebungen:

- Gazebo ermöglicht die Erstellung komplexer 3D-Umgebungen (z.B. Innenräume, Außenbereiche) mit verschiedenen Objekten, Materialien und Lichtverhältnissen.

Steuerung von Robotern in Gazebo

ROS-Integration:

- Gazebo ist eng mit dem Robot Operating System (ROS) integriert. ROS-Knoten können zur Steuerung der Roboter in der Simulation verwendet werden.

Prof. Dr. Daniel Gaida
Professor für Cyber-Physische Systeme
Fakultät für Informatik und Ingenieurwissenschaften - Institut für Informatik
Technology Arts Sciences TH Köln

Steuerung von Robotern in Gazebo

ROS-Integration:

- Gazebo ist eng mit dem Robot Operating System (ROS) integriert. ROS-Knoten können zur Steuerung der Roboter in der Simulation verwendet werden.

Algorithmenentwicklung:

- Gazebo ermöglicht die Entwicklung und das Testen von Algorithmen für Navigation, Planung, Bildverarbeitung und andere Robotikaufgaben.

Steuerung von Robotern in Gazebo

ROS-Integration:

- Gazebo ist eng mit dem Robot Operating System (ROS) integriert. ROS-Knoten können zur Steuerung der Roboter in der Simulation verwendet werden.

Algorithmenentwicklung:

- Gazebo ermöglicht die Entwicklung und das Testen von Algorithmen für Navigation, Planung, Bildverarbeitung und andere Robotikaufgaben.

Datenaufzeichnung und -analyse:

- Sensordaten und Roboterbewegungen können während der Simulation aufgezeichnet und anschließend analysiert werden.

Lernziel

Die Studierenden können grundlegend mit ROS2 umgehen, indem Sie ROS2 Nodes, Topics, Services, Packages, … nutzen und erstellen können, um später Komponenten für Cyber-Physische Systeme mit ROS2 entwickeln zu können.

Nächste Vorlesung und Fragen

- Modellierung von CPS
- Fragen?